

Short Video Recommender System: Report

Name: Aziz Zeghal

Login: aziz.zeghal

Subject: REMA

Promotion: SCIA 2026



Repository: FinalProject_2025_aziz.zeghal

Content

1	Introduction	1
1.1	Context	1
1.2	Dataset	1
1.3	Exploratory data analysis	1
2	ALS model	2
2.1	Description	2
2.2	Evaluation	2
2.3	Conclusion	2
2.4	Possible extensions	3
3	Two-Tower neural network	4
3.1	Description	4
3.2	Evaluation	4
3.3	Conclusion	4
3.4	Possible Extensions	5
4	Cosine similarity model	6
4.1	Description	6
4.2	Evaluation	6
4.3	Conclusion	6
4.4	Possible extensions	7
5	Conclusion	8

1. Introduction

1.1 Context

This project's goal is to create a recommender system for short videos. It is part of the REMA course at EPITA, and it aims to provide a personalized recommendation system for users based on their interactions with short videos.

Different approaches were tested: collaborative filtering, content-based filtering, and hybrid methods. The main focus was the *Two-Tower neural network* model.

1.2 Dataset

The dataset used in this project is the KuaiRec dataset, which contains user interactions with short videos. The user-item interactions are stored in the *small_matrix.csv* (99.6% density) and the *big_matrix.csv* (16.3% density) files.

For more information about the dataset, please refer to the KuaiRec dataset website.

1.3 Exploratory data analysis

The EDA was performed on all the elements of the dataset. For more detail, please refer to the *EDA.ipynb* notebook, which includes heatmaps and additional analyses.

Overall, the KuaiRec dataset provides a rich and diverse set of user-video interactions, with both dense and sparse matrices. User engagement patterns vary by time of day and are more balanced in the small matrix, while the big matrix reveals greater variability and sparsity.

2. ALS model

2.1 Description

The Alternating Least Squares (ALS) model was initially implemented using the *implicit* library, and later changed to *pyspark.ml.recommendation* for better scalability. This library enables hyperparameter tuning and model evaluation using cross-validation.

ALS is a collaborative filtering algorithm. It works by factorizing the user-item interaction matrix into two lower-dimensional matrices: one for users and one for items. The goal is to minimize the difference between the original interaction matrix and the product of the two lower-dimensional matrices.

Considering that we only have implicit feedback, ALS can work well. We will not use demographic data for this simple model. This algorithm is mostly used for sparse datasets.

2.2 Evaluation

Evaluation is a used feedback for the model training. We will use the *RegressionEvaluator* class from *pyspark.ml.evaluation* to evaluate the model with the rmse metric.

This model uses almost no feature to none, and is pretty basic. *kuaiREC_caption_category.csv* was used for evaluation to get the caption of the recommended videos.

The evaluation was performed using standard top-K ranking metrics. For each user in the test set, the model generates a ranked list of top-10 recommended items. These recommendations are compared against the actual items the user interacted with. NDCG, recall and precision were computed. These metrics were averaged across all users to obtain the final performance scores.

2.3 Conclusion

Problems were encountered when training the ALS model with the *big_matrix.csv* dataset. The *small_matrix.csv* dataset was used instead.

The ALS model served as a baseline collaborative filtering method for the KuaiRec short video recommendation task. However, its performance was limited by several factors:

- **Data Constraints:** Due to computational issues, only *small_matrix.csv* could be used for training, restricting the model's ability to learn from the full diversity and sparsity of the dataset.
- **Model Simplicity:** The model did not leverage any side information or advanced features, relying solely on user-item interaction data.

2.4 Possible extensions

Given the limitations of ALS and the results observed, several possible extensions could be considered to improve performance:

- **More features:** Adding more video features and user features.
- **Switch to Larger, Sparser Matrices:** Address the scalability and memory issues that prevented the use of *big_matrix.csv*. This could involve distributed training, more efficient data pipelines, or dimensionality reduction before factorization.
- **Advanced Matrix Factorization Techniques:** Explore more sophisticated matrix factorization approaches, such as Bayesian Personalized Ranking (BPR) or Weighted Matrix Factorization, which are often better suited for implicit feedback and large-scale data.

Overall, ALS is a good starting point and a classic collaborative filtering approach, but its performance is limited by the computing resources.

3. Two-Tower neural network

3.1 Description

The Two-Tower Neural Network model was more promising than the ALS model, and easier to use. It was implemented using the Keras library. The model uses a subset of *big_matrix.csv* for training and a subset of *small_matrix.csv* for evaluation.

The Two-Tower model is an embedding-based neural network architecture primarily used for large-scale retrieval tasks, such as search and recommendation systems. In this approach, two separate neural networks-referred to as "towers"-are used to independently encode user and item features into dense vector representations. These embeddings are then combined, typically via concatenation or similarity computation, to predict user-item interactions.

The model leverages both user and item metadata, including features such as content tags, user activity history, and video attributes, which are processed through dedicated preprocessing pipelines. It focused mostly on using video features, and PCA was used to reduce the dimensionality as it was too much for the graphic card.

3.2 Evaluation

The evaluation of the Two-Tower model was performed using standard recommender system metrics. During training, the model predicts a continuous score (watch ratio), which is compared against the ground truth using metrics such as mean absolute error (MAE).

After the training, to assess the quality of these rankings, we compute the NDCG@K, which evaluates how well the top-ranked items align with the ground-truth relevance labels. In addition, we binarize the predicted scores using a relevance threshold to calculate Precision@K and Recall@K, providing insight into the accuracy and completeness of the recommendations.

The final evaluation is based on comparing the predicted ranking matrix to the true relevance matrix across users, allowing us to measure the model's effectiveness in prioritizing relevant content.

3.3 Conclusion

The Two-Tower model provided a scalable and flexible baseline for the KuaiRec short video recommendation task. Unlike simple collaborative filtering models, it effectively utilized both user and item side information, enabling richer representations and improved personalization.

The model successfully handled the larger, sparser *big_matrix.csv* dataset, demonstrating its scalability. By incorporating additional features, the model moved beyond

the limitations of interaction-only approaches.

However, challenges remain in the model learning, and tuning neural network hyperparameters to further improve ranking performance.

3.4 Possible Extensions

Several ways could be explored to enhance the Two-Tower model's performance:

- **Feature Expansion:** Integrate more granular user and item features.
- **Advanced Architectures:** Experiment with deeper or more complex neural network structures within each tower, or introduce attention mechanisms to better capture user-item relationships.
- **Negative Sampling and Loss Functions:** Employ advanced sampling strategies or ranking-based loss functions (e.g., Bayesian Personalized Ranking) to better optimize for implicit feedback scenarios.

Overall, the Two-Tower model establishes a strong foundation for recommendation, with substantial room for further refinement and improvement.

4. Cosine similarity model

4.1 Description

This model is a naive test to see if the cosine similarity between the video features is a good metric to use for recommendation.

The initial approach involved training and testing on *small_matrix* and *big_matrix* separately. We then trained on *big_matrix* and tested on *small_matrix*.

Cosine similarity is a metric used to measure how similar two vectors are. Users and items can be represented as vectors in a multi-dimensional space, and cosine similarity can be used to find the most similar items to a given item or the most similar users to a given user.

Unlike collaborative filtering approaches that rely on user-item interaction patterns, this method leverages explicit item and user features, making it suitable even when interaction data is sparse.

4.2 Evaluation

The system is evaluated as a binary classification problem. Given a set of recommended items, we assess whether the user genuinely liked the item. This is determined by checking if the user watched more than a defined threshold percentage (for example, 70%) of the video.

For each recommendation, the label is:

- **Positive (1):** if the user watched more than the threshold
- **Negative (0):** otherwise

This is the same method as the *Two-Tower* model.

4.3 Conclusion

This model is very simple, yet its performance is highly sensitive to data sparsity.

On the denser *small_matrix*, the results are generally better, as it is easier to identify meaningful user similarities and to verify user-item interactions. In contrast, performance drops significantly on the sparser *big_matrix*. This decline is not necessarily due to poor recommendations, but rather to limitations in the evaluation process: a recommendation can only be validated if the user has actually interacted with the item in the test set. As a result, many relevant suggestions may go unrecognized.

Consequently, the model may appear to perform worse than it actually does.

One possible solution is to train on *big_matrix* and use *small_matrix* for label verification. However, this approach did not resolve the issue.

4.4 Possible extensions

This model can be improved with:

- **More features:** Adding more video features (e.g., *kuareic_caption_category*) and user features (e.g., *social_network*).
- **History-based recommendations:** Prioritizing items that are similar to those in the user's watch history.
- **Friend weighting:** Weighting recommendations based on friendship connections.
- **Popularity scoring:** Incorporating item popularity into the item feature vector. More popular items may have a higher chance of being watched.

5. Conclusion

Overall, this project was a great opportunity to learn about recommender systems and to gain hands-on experience with several modeling approaches.

Among the methods implemented, the Two-Tower model showed the most promise, offering a flexible framework for incorporating both user and item features.

However, it still requires further refinement and tuning to reach its full potential. The ALS model served as a solid baseline and highlighted the challenges posed by data sparsity and implicit feedback.

Meanwhile, the cosine similarity approach, though simple, demonstrated that even basic techniques can yield meaningful recommendations under certain conditions.

Throughout the project, I encountered a variety of challenges, particularly related to data preprocessing, evaluation limitations, and model tuning.

I am grateful for the opportunity provided by this course, which helped bridge the gap between theory and practice and sparked a strong interest in further exploring this domain.